ENSEEIHT

UE : Ingénierie Dirigée par les Modèles

Spécification et Validation de Circuits Électroniques

Projet IDM 2025–2026

Groupe L12-5

Membres du groupe :

CHAKIR Yassir
RJA-FALLAH Mohamed Amine
EL-AMRANI Abdelhamid
DIOUANE Walid

Encadrant : M. Marc Pantel

Année universitaire : 2025 – 2026

Table des matières

1 Introduction **4**
1.1 Contexte général . . . 4
1.2 Objectifs du projet . . . 4
1.3 Organisation du rapport . . . 4

2 Conception des méta-modèles **5**
2.1 Méta-modèle Catalogue . . . 5
2.1.1 Analyse du domaine . . . 5
2.1.2 Diagramme de classes . . . 5
2.1.3 Description des concepts . . . 5
2.1.4 Hiérarchie des contraintes . . . 6
2.1.5 Justification des choix . . . 6
2.2 Méta-modèle Netlist . . . 6
2.2.1 Rôle de la netlist . . . 6
2.2.2 Diagramme de classes . . . 7
2.2.3 Description des concepts . . . 7
2.2.4 Gestion des références croisées . . . 8
2.3 Méta-modèle Layout . . . 8
2.3.1 Représentation physique du circuit . . . 8
2.3.2 Diagramme de classes . . . 9
2.3.3 Description des concepts . . . 9
2.3.4 Gestion des couches . . . 10
2.4 Synthèse et interdépendances . . . 10

3 Validation par contraintes **11**
3.1 Architecture de la validation . . . 11
3.1.1 Principe du patron Switch . . . 11
3.1.2 Collecte des résultats . . . 11
3.2 Exemple : validation du Catalogue . . . 11
3.3 Tableau récapitulatif des contraintes . . . 11

4 Transformations modèle vers texte **14**
4.1 Principe de génération . . . 14
4.1.1 Format DOT et Graphviz . . . 14
4.2 Visualisation du Catalogue . . . 14
4.3 Visualisation de la Netlist . . . 14
4.4 Génération de la BOM . . . 15
4.5 Visualisation du Layout . . . 15

5 Éditeurs graphiques Sirius **17**
5.1 Éditeur du Catalogue . . . 17
5.1.1 Représentations graphiques . . . 17
5.1.2 Palette d'outils . . . 17
5.2 Éditeur de la Netlist . . . 18
5.2.1 Représentation des circuits . . . 18

5.2.2 Gestion des connexions 18

6 Description du rendu **19**
6.1 Dossier Catalogue 19
6.1.1 Méta-modèle et génération 19
6.1.2 Validation 19
6.1.3 Éditeur Sirius 19
6.1.4 Transformation M2T 19
6.1.5 Modèle exemple et documentation 19
6.2 Dossier Netlist 20
6.2.1 Méta-modèle et génération 20
6.2.2 Validation 20
6.2.3 Éditeur Sirius 20
6.2.4 Transformations M2T 20
6.2.5 Modèle exemple et documentation 20
6.3 Dossier Layout 20
6.3.1 Méta-modèle et génération 20
6.3.2 Validation 21
6.3.3 Éditeur Sirius 21
6.3.4 Transformation M2T 21
6.3.5 Modèle exemple et documentation 21
6.4 Couverture fonctionnelle 21

7 Conclusion **22**
7.1 Bilan technique 22
7.2 Difficultés rencontrées 22
7.3 Bilan personnel 22

Table des figures

2.1 Diagramme de classes du méta-modèle Catalogue 5
2.2 Diagramme de classes du méta-modèle Netlist 7
2.3 Diagramme de classes du méta-modèle Layout 9

4.1 Visualisation du catalogue générée via transformation M2T 14
4.2 Visualisation de la netlist du circuit RLC série 14
4.3 Bill of Materials du circuit RLC série 15
4.4 Visualisation du layout du circuit RLC série 16

5.1 Éditeur Sirius du catalogue de composants 17
5.2 Éditeur Sirius de la netlist – Circuit RLC série 18

Chapitre 1

Introduction

1.1 Contexte général

L'électronique embarquée constitue aujourd'hui un domaine en pleine expansion, porté par la miniaturisation croissante des composants et l'amélioration continue des processus de fabrication. Dans ce contexte, les systèmes dits d'« embarqué extrême » – satellites, drones sous-marins, dispositifs autonomes – posent des défis particuliers. Une fois déployés, ces systèmes ne peuvent être maintenus qu'au travers de passerelles limitées, ce qui rend leur robustesse et leur résilience absolument critiques.

Face à cette problématique, les ingénieurs électroniciens doivent concevoir des circuits en respectant un ensemble de contraintes dont la satisfaction garantit l'absence de certaines catégories de fautes. Cependant, cette vérification reste largement manuelle : l'ingénieur réalise un design, puis vérifie la validité des contraintes visuellement, une approche laborieuse et sujette aux erreurs humaines.

1.2 Objectifs du projet

Ce projet s'inscrit dans le cadre de l'unité d'enseignement d'Ingénierie Dirigée par les Modèles. Son objectif principal est double. D'une part, il s'agit de nous confronter aux technologies EMF et Sirius dans un contexte réaliste. D'autre part, le projet vise à approfondir notre maîtrise des transformations de modèles et de la validation par contraintes.

Concrètement, nous devons développer une suite d'outils permettant de modéliser des circuits électroniques selon trois niveaux d'abstraction distincts : le catalogue de composants, la netlist (schéma logique des connexions) et le layout (disposition physique sur circuit imprimé). Chaque niveau possède ses propres contraintes et règles de cohérence qu'il convient de formaliser puis de vérifier automatiquement.

1.3 Organisation du rapport

Ce rapport présente notre démarche de conception et d'implémentation. Après cette introduction, le chapitre 2 détaille la conception des trois méta-modèles constituant le cœur de notre solution. Le chapitre 3 expose les mécanismes de validation développés en Java. Le chapitre 4 traite des transformations modèle-vers-texte pour la génération de visualisations. Le chapitre 5 présente les éditeurs graphiques réalisés avec Sirius. Le chapitre 6 décrit en détail le contenu du rendu. Enfin, la conclusion dresse un bilan critique du projet.

Chapitre 2

Conception des méta-modèles

2.1 Méta-modèle Catalogue

2.1.1 Analyse du domaine

Le catalogue constitue la bibliothèque de référence des composants électroniques disponibles. Chaque composant y est décrit par ses caractéristiques intrinsèques : nom, fabricant, empreinte physique et ports de connexion. Au-delà de ces attributs descriptifs, le catalogue doit également capturer les contraintes associées à chaque composant, qu'il s'agisse de contraintes géométriques, de redondance ou de compatibilité.

2.1.2 Diagramme de classes

La figure 2.1 présente le diagramme de classes Ecore de notre méta-modèle Catalogue.

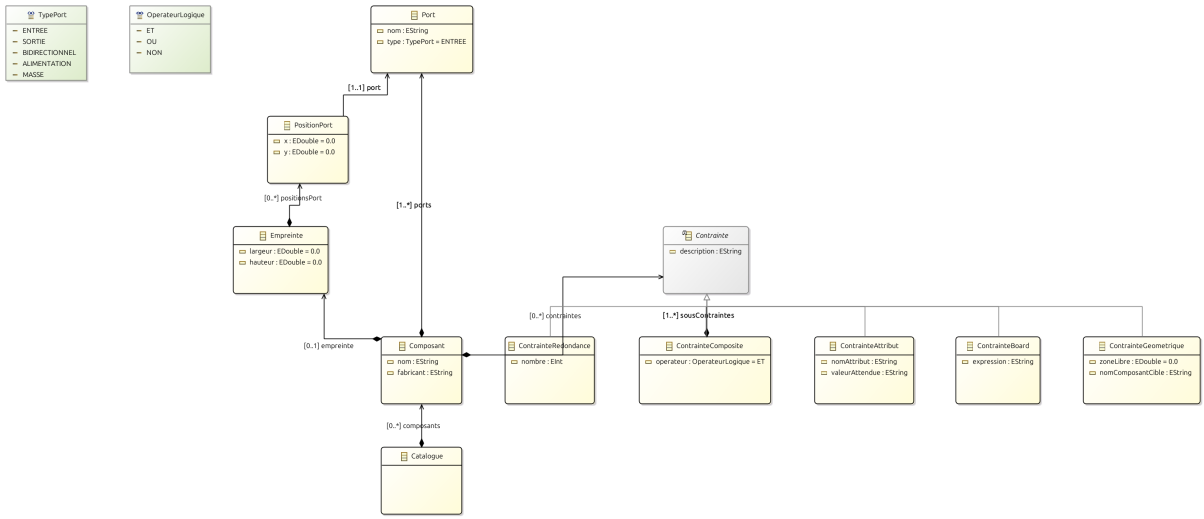

FIGURE 2.1 – Diagramme de classes du méta-modèle Catalogue

2.1.3 Description des concepts

Catalogue représente l'élément racine contenant l'ensemble des composants. Cette classe sert de conteneur principal et permet de regrouper logiquement les composants par famille ou par projet.

Composant modélise un composant électronique avec ses attributs `nom` (obligatoire) et `fabricant` (optionnel). Un composant possède une relation de composition avec ses ports (multiplicité 1..*) et peut optionnellement référencer une empreinte. Il peut également être associé à plusieurs contraintes.

Port représente un point de connexion du composant. Chaque port est caractérisé par son nom et son type. L'énumération `TypePort` définit cinq valeurs possibles : ENTREE, SORTIE, BI-

DIRECTIONNEL, ALIMENTATION et MASSE. Cette typologie permet de vérifier la cohérence des connexions lors de la constitution de la netlist.

Empreinte décrit l'encombrement physique du composant sur le PCB via ses attributs `largeur` et `hauteur` (en millimètres). Elle contient également les positions des ports sur cette empreinte via la classe `PositionPort`.

PositionPort associe un port à ses coordonnées (x, y) sur l'empreinte. Cette classe permet de positionner précisément chaque broche du composant pour le routage ultérieur.

2.1.4 Hiérarchie des contraintes

Un aspect particulièrement intéressant de ce méta-modèle concerne la modélisation des contraintes. Nous avons choisi d'utiliser le patron de conception Composite pour permettre la combinaison flexible de contraintes élémentaires.

La classe abstraite **Contrainte** sert de base commune avec un attribut optionnel `description`. Cinq sous-classes spécialisées en héritent :

ContrainteComposite combine plusieurs sous-contraintes via un opérateur logique (ET, OU, NON définis dans l'énumération `OperateurLogique`). Cette classe référence récursivement d'autres contraintes, permettant de construire des expressions complexes.

ContrainteGeometrique spécifie une zone libre (`zoneLibre`) à maintenir autour du composant, éventuellement par rapport à un composant cible identifié par `nomComposantCible`.

ContrainteRedondance impose qu'un composant soit présent en plusieurs exemplaires via l'attribut `nombre`.

ContrainteBoard exprime une contrainte relative aux caractéristiques du board via une `expression` textuelle.

ContrainteAttribut vérifie qu'un attribut du layout (`nomAttribut`) possède une `valeurAttendue` spécifique.

2.1.5 Justification des choix

Le choix de séparer l'empreinte du composant plutôt que d'intégrer directement les dimensions dans la classe Composant répond à un besoin de flexibilité.

L'utilisation du patron Composite pour les contraintes offre une grande expressivité. On peut ainsi définir des contraintes complexes du type « (distance > 10mm ET température < 80°C) OU composant de refroidissement présent » par simple composition d'éléments primitifs.

2.2 Méta-modèle Netlist

2.2.1 Rôle de la netlist

La netlist constitue le schéma logique du circuit, décrivant quels ports de quels composants sont connectés entre eux, indépendamment de toute considération d'implantation physique. C'est l'équivalent d'un schéma électronique classique où l'on voit les composants reliés par des fils, sans se soucier de leur disposition réelle.

2.2.2 Diagramme de classes

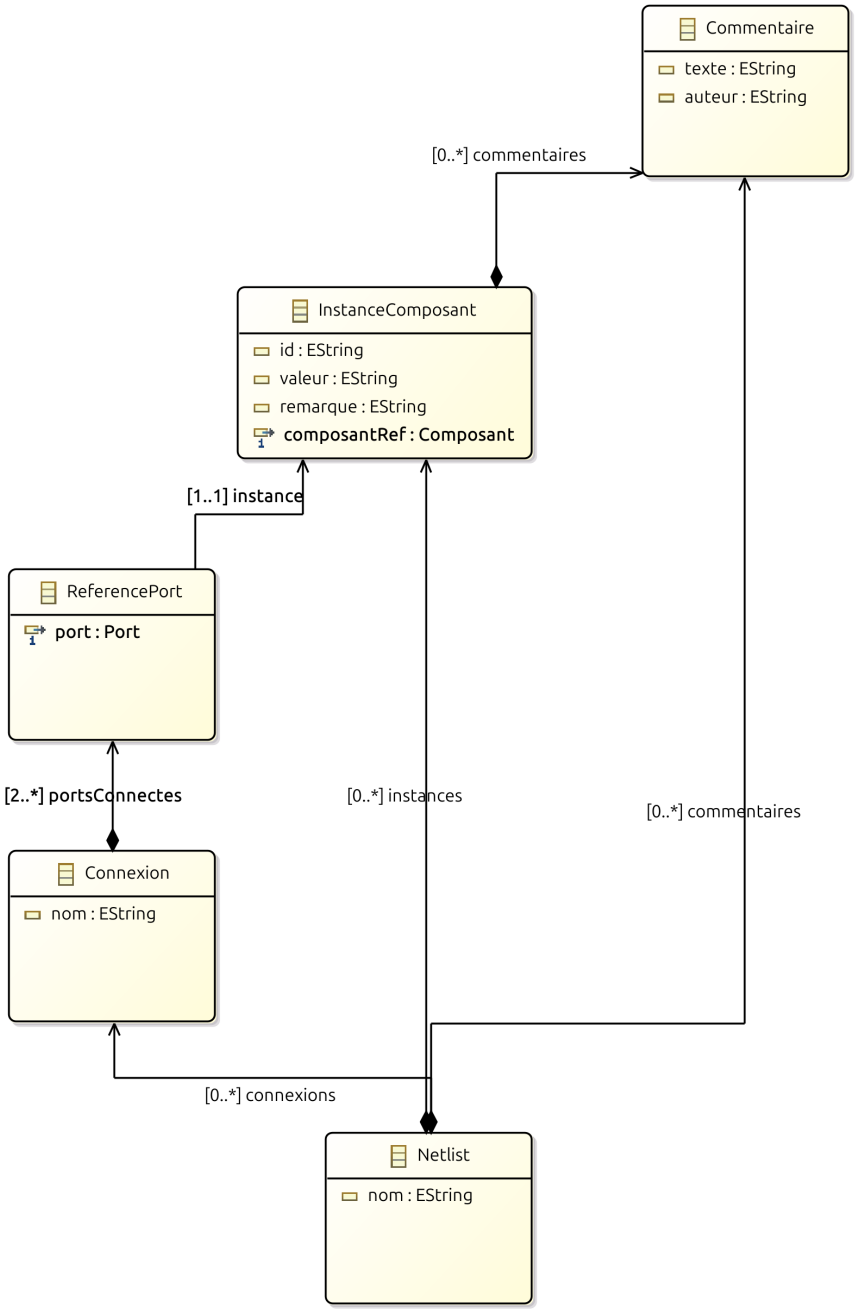

FIGURE 2.2 – Diagramme de classes du méta-modèle Netlist

2.2.3 Description des concepts

Netlist est l'élément racine identifié par un nom. Il contient les instances de composants, les connexions entre ces instances, et éventuellement des commentaires globaux.

InstanceComposant représente une occurrence concrète d'un composant du catalogue dans le circuit. L'attribut `id` fournit un identifiant unique (ex : R1, C2, IC3), tandis que `valeur` permet de spécifier une valeur spécifique (100Ω, 10μF, etc.). L'attribut `remarque` autorise l'ajout d'annotations libres. La référence `composantRef` établit le lien avec la définition du composant dans

le catalogue – il s'agit d'une référence croisée inter-modèles.

Connexion modélise un nœud électrique, c'est-à-dire un ensemble de ports électriquement reliés. Le nom est optionnel mais utile pour identifier les signaux importants (VDD, GND, CLK, etc.). Une connexion regroupe au minimum deux références de ports.

ReferencePort permet de désigner un port spécifique d'une instance donnée. Cette indirection est nécessaire car un même composant peut être instancié plusieurs fois, et il faut pouvoir distinguer le port A de R1 du port A de R2.

Commentaire autorise l'ajout d'annotations textuelles avec mention de l'auteur. Les commentaires peuvent être attachés soit au niveau global de la netlist, soit à une instance particulière.

2.2.4 Gestion des références croisées

Un point technique important concerne la gestion des références croisées entre méta-modèles. La référence `composantRef` de type `Composant` implique que le méta-modèle Netlist importe le méta-modèle Catalogue. Lors de la génération du code EMF, nous avons dû veiller à générer d'abord les sources du catalogue(on a fait un "load ressources"), puis à référencer correctement le genmodel correspondant lors de la création du genmodel de la netlist.

2.3 Méta-modèle Layout

2.3.1 Représentation physique du circuit

Le layout représente l'implantation physique du circuit sur un ou plusieurs PCB (Printed Circuit Board). Il s'agit de décrire où sont placés les composants et comment les pistes conductrices les relient, en tenant compte de la structure multicouche des cartes modernes.

2.3.2 Diagramme de classes

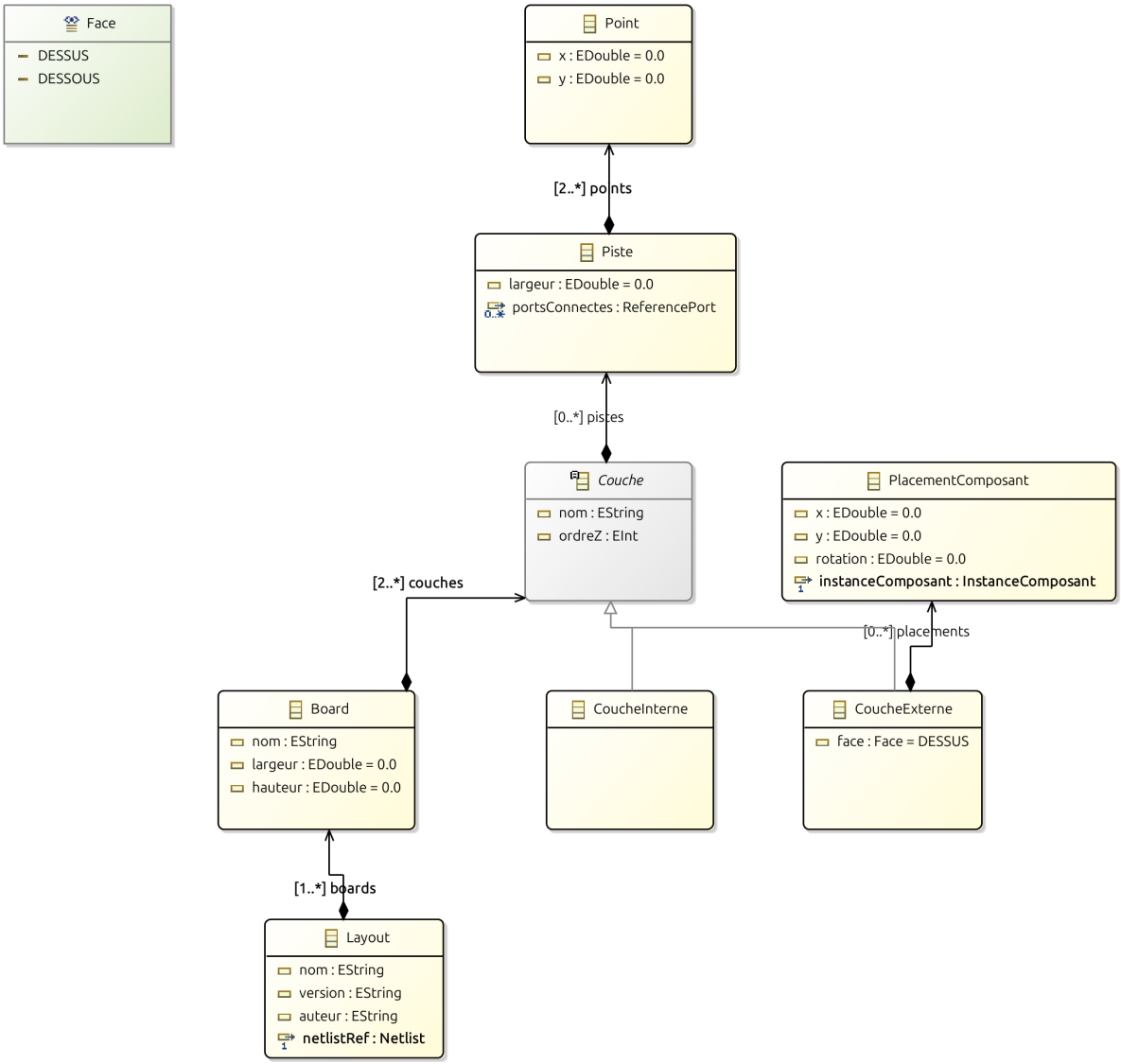

FIGURE 2.3 – Diagramme de classes du méta-modèle Layout

2.3.3 Description des concepts

Layout est l'élément racine contenant les métadonnées du projet (`nom`, `version`, `auteur`) et référençant la netlist correspondante via `netlistRef`. Un layout peut contenir plusieurs boards.

Board représente une carte physique avec ses dimensions (`largeur`, `hauteur`) et ses couches. La contrainte de multiplicité 2..* sur les couches traduit le fait qu'un PCB possède au minimum une face supérieure et une face inférieure.

Couche est une classe abstraite modélisant une couche du PCB. Chaque couche possède un nom, un ordre Z (position dans l'empilement) et peut contenir des pistes conductrices.

CoucheExterne est une spécialisation représentant les faces visibles du PCB (DESSUS ou DESSOUS selon l'énumération `Face`). Seules les couches externes peuvent accueillir des placements de composants, ceux-ci étant physiquement montés en surface.

CoucheInterne est une spécialisation pour les couches de routage internes, utilisées uniquement pour le passage de pistes sans composants.

PlacementComposant associe une instance de composant à une position (x, y) et une rotation sur une couche externe. La référence vers `InstanceComposant` du méta-modèle Netlist constitue une seconde référence croisée.

Piste représente une trace conductrice définie par sa largeur et une séquence d'au moins deux points. La référence optionnelle vers des `ReferencePort` permet d'indiquer quels ports la piste relie.

Point est un simple couple de coordonnées (x, y) décrivant la géométrie d'une piste.

2.3.4 Gestion des couches

La distinction entre couches internes et externes répond aux contraintes physiques de fabrication des PCB. Un PCB standard à 4 couches comprend typiquement une couche externe dessus, deux couches internes, et une couche externe dessous. L'attribut `ordreZ` permet d'ordonnancer correctement ces couches pour la visualisation et l'export.

2.4 Synthèse et interdépendances

Les trois méta-modèles forment une chaîne de dépendances cohérente. Le Catalogue, purement descriptif, ne dépend de rien. La Netlist référence le Catalogue pour instancier des composants. Le Layout référence la Netlist pour placer les instances et vérifier la cohérence du routage. Cette organisation permet une évolution indépendante de chaque partie tout en maintenant la traçabilité entre les différents niveaux d'abstraction.

Chapitre 3

Validation par contraintes

La validation constitue un aspect fondamental de notre suite d'outils. Elle permet de garantir la cohérence des modèles et de détecter les erreurs de conception avant qu'elles ne se propagent aux étapes ultérieures. Plutôt que d'utiliser OCL, nous avons opté pour une validation en Java utilisant le patron Switch généré par EMF.

3.1 Architecture de la validation

3.1.1 Principe du patron Switch

EMF génère automatiquement une classe utilitaire `<Metamodel>Switch<T>` pour chaque méta-modèle. Cette classe implémente un mécanisme de double dispatch permettant d'appeler une méthode spécifique selon le type réel de l'objet visité. Nos validateurs héritent de ces classes et redéfinissent les méthodes `case<ClassName>` appropriées.

3.1.2 Collecte des résultats

Pour collecter les résultats de validation, nous avons défini une classe `ValidationResult` offrant des méthodes pour enregistrer les erreurs associées à des éléments du modèle, vérifier conditionnellement des prédicats, et consulter la liste des erreurs détectées. Cette approche permet de collecter toutes les erreurs en une passe plutôt que de s'arrêter à la première.

3.2 Exemple : validation du Catalogue

Pour illustrer notre approche, prenons l'exemple de la validation d'un `Composant`. Le validateur vérifie que le nom est non vide, que le fabricant (s'il est défini) n'est pas une chaîne vide, que le composant possède au moins un port, et que l'empreinte (si présente) est valide. Ensuite, il visite récursivement les ports et les contraintes pour les valider à leur tour.

La validation des `ContrainteComposite` présente une complexité particulière due à leur nature récursive. Nous vérifions notamment que l'opérateur NON possède exactement une sous-contrainte, que les opérateurs ET et OU possèdent au moins deux sous-contraintes, et qu'il n'existe pas de cycles dans la structure (une contrainte qui se référencerait elle-même directement ou indirectement). La détection de cycles utilise un ensemble des nœuds visités : si un nœud est rencontré deux fois sur un même chemin de descente, un cycle existe.

3.3 Tableau récapitulatif des contraintes

Le tableau présente les contraintes vérifiées pour le méta-modèle Catalogue.

Élément	**Code**	**Description**
Composant	C2.1	Le nom ne doit pas être vide
	C2.2	Le fabricant ne doit pas être vide s'il est défini
	C2.4	Au moins un port requis
Port	C4.1	Le nom ne doit pas être vide
	C4.2	Le nom doit être unique dans le composant
	C4.3	Le type doit être défini
Empreinte	C3.1	La largeur doit être strictement positive
	C3.2	La hauteur doit être strictement positive
PositionPort	C5.1	Doit référencer un port existant
	C5.2	Coordonnées dans les limites de l'empreinte
	C5.3	Le port doit appartenir au même composant
ContrainteComposite	C7.1	L'opérateur logique doit être défini
	C7.3	NON : exactement 1 sous-contrainte
	C7.4	ET/OU : au moins 2 sous-contraintes
	C7.6	Pas de cycle dans la hiérarchie
ContrainteGeometrique	C8.1	Zone libre positive ou nulle
	C8.3	Composant cible doit exister
ContrainteRedondance	C9.1	Nombre strictement positif

TABLE 3.1 – Contraintes de validation du méta-modèle Catalogue

Le tableau présente les contraintes vérifiées pour le méta-modèle Netlist.

Élément	Code	Description
Netlist	C1.1 C1.2	Le nom ne doit pas être vide s'il est défini Le nom doit respecter le format identifiant
InstanceComposant	C2.1 C2.2 C2.3 C2.4	L'ID ne doit pas être vide L'ID doit respecter le format identifiant L'ID doit être unique dans la netlist Doit référencer un composant du catalogue
Connexion	C3.1 C3.2 C3.3	Le nom ne doit pas être vide s'il est défini Au moins 2 ports connectés requis Pas de port en double dans la connexion
ReferencePort	C4.1 C4.2 C4.3	Doit référencer une instance existante Doit référencer un port existant L'instance doit appartenir à la même netlist
Commentaire	C5.1 C5.2	Le texte ne doit pas être vide s'il est défini L'auteur ne doit pas être vide s'il est défini

TABLE 3.2 – Contraintes de validation du méta-modèle Netlist

Le tableau présente les contraintes vérifiées pour le méta-modèle Layout.

Élément	Code	Description
Layout	C1.3 C1.4	Doit référencer une netlist Au moins un board requis
Board	C2.2 C2.3 C2.4 C2.5 C2.6	Largeur strictement positive Hauteur strictement positive Au moins 2 couches requises Noms de couches uniques Valeurs ordreZ uniques
CoucheExterne	C4.1 C4.2	La face doit être définie (DESSUS/DESSOUS) Placements dans les limites du board
PlacementComposant	C6.1 C6.2 C6.3 C6.5	Doit référencer une instance Coordonnées positives ou nulles Rotation entre 0 et 360 degrés Instance placée une seule fois par board
Piste	C7.1 C7.2 C7.3 C7.4	Largeur strictement positive Au moins 2 points requis Points dans les limites du board Pas de points en double
Point	C8.1	Coordonnées positives ou nulles

TABLE 3.3 – Contraintes de validation du méta-modèle Layout

Chapitre 4

Transformations modèle vers texte

Les transformations Model-to-Text (M2T) permettent de générer des artefacts textuels à partir des modèles. Nous avons développé des générateurs produisant des fichiers au format DOT pour la visualisation graphique des modèles.

4.1 Principe de génération

4.1.1 Format DOT et Graphviz

Le format DOT est un langage de description de graphes interprété par Graphviz. Un fichier DOT décrit les nœuds et arêtes d'un graphe, avec leurs attributs visuels. Graphviz se charge ensuite du placement automatique des éléments et de l'export vers différents formats (PDF, PNG, SVG).

4.2 Visualisation du Catalogue

Pour le catalogue, chaque composant est représenté comme un sous-graphe contenant sa description, son empreinte et ses ports. Les informations affichées comprennent le nom et le fabricant, les dimensions de l'empreinte, la liste des ports avec leur type, et le nombre de contraintes définies.

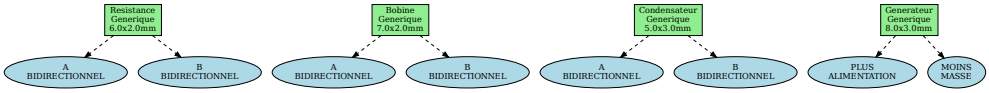

FIGURE 4.1 – Visualisation du catalogue générée via transformation M2T

4.3 Visualisation de la Netlist

La netlist est visualisée comme un graphe où les nœuds représentent les instances de composants et les arêtes les connexions. Cette représentation permet de vérifier visuellement la topologie du circuit.

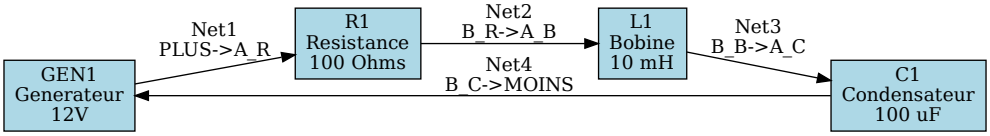

FIGURE 4.2 – Visualisation de la netlist du circuit RLC série

4.4 Génération de la BOM

Une transformation spécifique génère la Bill of Materials (nomenclature) sous forme tabulaire. La BOM récapitule pour chaque instance son identifiant, le composant de référence, sa valeur et les éventuelles remarques. Ce document est essentiel pour l'approvisionnement et la fabrication.

ID	Composant	Valeur	Fabricant	Remarque
GEN1	Generateur	12V	Generique	Source de tension
R1	Resistance	100 Ohms	Generique	Resistance de limitation
L1	Bobine	10 mH	Generique	Inductance
C1	Condensateur	100 uF	Generique	Capacite

Statistiques BOM	
Nombre total de composants:	4
Types de composants uniques:	4
Nombre de connexions:	4

FIGURE 4.3 – Bill of Materials du circuit RLC série

4.5 Visualisation du Layout

Le layout présente une complexité supplémentaire due à sa structure multicouche. Nous générons une vue par couche, représentant les placements de composants (pour les couches externes) et les pistes.

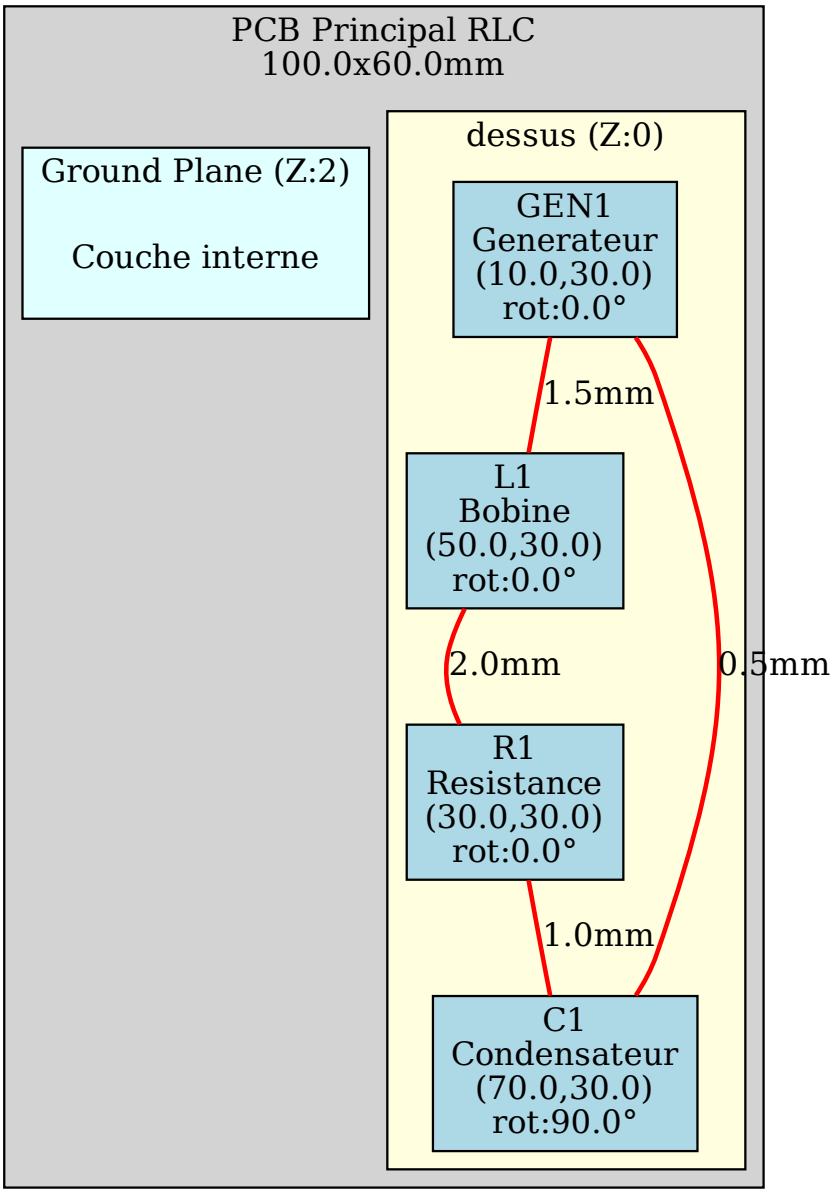

FIGURE 4.4 – Visualisation du layout du circuit RLC série

Chapitre 5

Éditeurs graphiques Sirius

5.1 Éditeur du Catalogue

5.1.1 Représentations graphiques

L'éditeur de catalogue représente chaque composant avec son empreinte, ses ports et ses contraintes. Nous avons défini des représentations spécifiques pour différents types de composants : le condensateur avec un symbole parallélépipédique à deux traits, le générateur avec un cercle marqué de la lettre G, la diode avec un triangle et une barre, la résistance avec un rectangle allongé, et la bobine avec une spirale caractéristique.

Les ports sont représentés comme des points positionnés sur l'empreinte selon leurs coordonnées réelles.

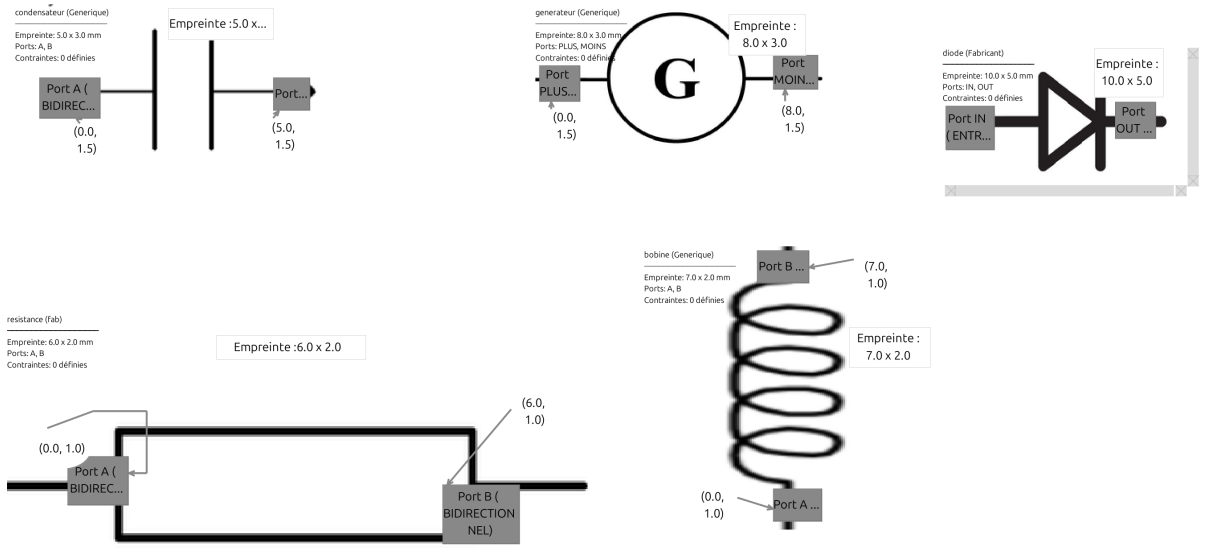

FIGURE 5.1 – Éditeur Sirius du catalogue de composants

5.1.2 Palette d'outils

Une palette permet à l'utilisateur de créer de nouveaux éléments : créer un composant (avec choix du type), ajouter un port à un composant, définir une empreinte, et ajouter une contrainte. Les propriétés de chaque élément sont éditables via la vue Properties d'Eclipse.

5.2 Éditeur de la Netlist

5.2.1 Représentation des circuits

L'éditeur de netlist permet de visualiser et d'éditer les circuits électroniques. Chaque instance de composant est représentée par son symbole électrique conventionnel avec son identifiant et sa valeur.

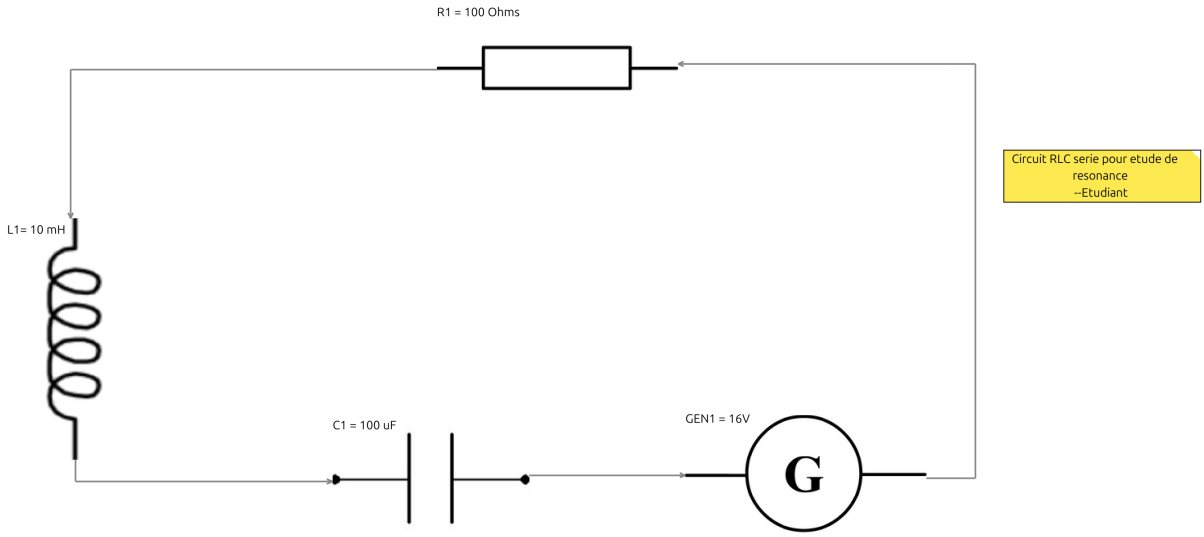

FIGURE 5.2 – Éditeur Sirius de la netlist – Circuit RLC série

5.2.2 Gestion des connexions

Les connexions sont représentées par des lignes reliant les ports. Un outil de création de connexion permet de tracer des liens entre ports en sélectionnant successivement le point de départ et d'arrivée. Les commentaires sont affichés sous forme de notes jaunes attachées aux éléments concernés.

Chapitre 6

Description du rendu

Ce chapitre décrit en détail l'organisation et le contenu du rendu fourni. Le projet est structuré en trois dossiers principaux correspondant aux trois méta-modèles développés : Catalogue, Netlist et Layout.

6.1 Dossier Catalogue

Le dossier `catalogue/` contient l'ensemble des fichiers relatifs au méta-modèle de catalogue de composants électroniques.

6.1.1 Méta-modèle et génération

— `catalogue.ecore` : méta-modèle Ecore définissant les concepts de Catalogue, Composant, Port, Empreinte, PositionPort et la hiérarchie des contraintes (Contrainte, ContrainteComposite, ContrainteGeometrique, ContrainteRedondance, ContrainteBoard, ContrainteAttribut).

6.1.2 Validation

— `CatalogueValidator.java` : validateur Java implémentant les contraintes de cohérence du catalogue (noms non vides, unicité des ports, dimensions positives, arité des opérateurs logiques, détection de cycles).

6.1.3 Éditeur Sirius

— `catalogue.odesign` : fichier de spécification de l'éditeur graphique Sirius définissant les représentations visuelles des composants et la palette d'outils.
— `catalogue.aird` : fichier de session Sirius contenant les diagrammes créés.
— `catalogue_sirius.png` : capture d'écran de l'éditeur Sirius en action.

6.1.4 Transformation M2T

— `todot.mtl` : template Acceleo pour la génération de fichiers DOT (Graphviz) à partir des modèles de catalogue.
— `catalogue.pdf` : visualisation PDF générée à partir du modèle exemple via Graphviz.

6.1.5 Modèle exemple et documentation

— `catalogue1.xmi` : modèle exemple contenant les composants de base (résistance, condensateur, bobine, générateur, diode) avec leurs empreintes et ports.
— `catalogue.png` : diagramme de classes du méta-modèle Catalogue.

6.2 Dossier Netlist

Le dossier `netlist/` contient l'ensemble des fichiers relatifs au méta-modèle de netlist (schéma logique des circuits).

6.2.1 Méta-modèle et génération

— `netlist.ecore` : méta-modèle Ecore définissant les concepts de Netlist, InstanceComposant, Connexion, ReferencePort et Commentaire, avec référence croisée vers le méta-modèle Catalogue.

6.2.2 Validation

— `NetlistValidator.java` : validateur Java implémentant les contraintes de cohérence de la netlist (unicité des identifiants, connexions valides, références correctes).

6.2.3 Éditeur Sirius

— `netlist.odesign` : fichier de spécification de l'éditeur graphique Sirius avec les symboles électriques standards (résistance, condensateur, bobine, etc.).
— `netlist.aird` : fichier de session Sirius contenant les diagrammes de circuits.
— `Circuit_RLC_serie_sirius.png` : capture d'écran de l'éditeur Sirius montrant le circuit RLC série.

6.2.4 Transformations M2T

— `todot.mtl` : template Acceleo pour la génération de fichiers DOT représentant la topologie du circuit.
— `netlistBOM.mtl` : template Acceleo pour la génération de la Bill of Materials (nomenclature des composants).
— `Circuit_RLC_Serie.pdf` : visualisation PDF du circuit RLC série générée via Graphviz.
— `Circuit_RLC_Serie_BOM.pdf` : nomenclature PDF du circuit listant tous les composants avec leurs valeurs.

6.2.5 Modèle exemple et documentation

— `Netlist1.xmi` : modèle exemple du circuit RLC série pour étude de résonance, comprenant une résistance (100Ω), un condensateur (100μF), une bobine (10mH) et un générateur (16V).
— `netlist.png` : diagramme de classes du méta-modèle Netlist.

6.3 Dossier Layout

Le dossier `layout/` contient l'ensemble des fichiers relatifs au méta-modèle de layout (implantation physique sur PCB).

6.3.1 Méta-modèle et génération

— `layout.ecore` : méta-modèle Ecore définissant les concepts de Layout, Board, Couche (CoucheExterne, CoucheInterne), PlacementComposant, Piste et Point, avec référence croisée vers le méta-modèle Netlist.

6.3.2 Validation

— `LayoutValidator.java` : validateur Java implémentant les contraintes de cohérence du layout (dimensions positives, placements dans les limites, unicité des couches, pistes valides).

6.3.3 Éditeur Sirius

— `layout.aird` : fichier de session Sirius pour la visualisation des layouts.

6.3.4 Transformation M2T

— `todot.mtl` : template Acceleo pour la génération de fichiers DOT représentant les couches du PCB.
— `Layout_Circuit_RLC_Serie.pdf` : visualisation PDF du layout du circuit RLC série montrant le placement des composants et les pistes.

6.3.5 Modèle exemple et documentation

— `Layout1.xmi` : modèle exemple du layout du circuit RLC série sur PCB double face avec placement des composants et routage des pistes.
— `layout.png` : diagramme de classes du méta-modèle Layout.

6.4 Couverture fonctionnelle

Le tableau 6.1 récapitule la couverture des fonctionnalités demandées dans la spécification.

Fonctionnalité	**État**	**Remarque**
F1.1 Métadonnées composants		Nom, fabricant implémentés
F1.2 Empreinte composant		Dimensions largeur/hauteur
F1.3 Ports sur empreinte		Positions (x,y) et types
F1.4 Contraintes composants		5 types + composite avec ET/OU/NON
F1.5 Interface catalogue		Éditeur Sirius avec palette
F2.1 Visualisation netlist		Via Sirius et génération PDF
F2.2 Interface graphique Sirius		Symboles électriques standards
F2.3 Commentaires		Sur instances et netlist
F2.4 Export BOM		Template netlistBOM.mtl
F3.1 Création layout		Via éditeur arborescent
F3.2 Structure multicouche		CoucheExterne + CoucheInterne
F3.3 Placement composants		Position (x,y) et rotation
F3.4 Pistes conductrices		Largeur et tracé par points
F3.5 Vérification cohérence	$\sim$	Validation structurelle OK
F3.6 Export images		Format PDF via Graphviz

TABLE 6.1 – Couverture fonctionnelle du projet

Chapitre 7

Conclusion

7.1 Bilan technique

Ce projet nous a permis de mettre en pratique les concepts fondamentaux de l'ingénierie dirigée par les modèles dans un contexte applicatif réaliste. Nous avons conçu trois méta-modèles interconnectés (Catalogue, Netlist, Layout) formant une chaîne cohérente de description des circuits électroniques, depuis la définition des composants jusqu'à leur implantation physique.

La validation par contraintes Java, offre une grande flexibilité et permet d'exprimer des vérifications complexes comme la détection de cycles ou les contrôles géométriques. Les transformations M2T vers le format DOT produisent des visualisations exploitables pour la revue et la documentation. Enfin, les éditeurs Sirius apportent l'ergonomie nécessaire à une utilisation quotidienne par des non-informaticiens.

7.2 Difficultés rencontrées

La gestion des références croisées entre méta-modèles a constitué un défi technique notable. L'ordre de génération des genmodels et la configuration des dépendances ont nécessité plusieurs itérations avant d'aboutir à une solution fonctionnelle. En particulier, les erreurs de type « Unresolved proxy » lors du chargement de modèles référençant d'autres modèles nous ont demandé un temps conséquent de débogage.

La conception des éditeurs Sirius s'est également révélée plus complexe que prévu. La documentation disponible, bien que complète, manque parfois d'exemples concrets pour des cas d'usage avancés.

Enfin, la synchronisation entre les différents niveaux (catalogue, netlist, layout) lors de modifications pose des questions de cohérence que nous n'avons que partiellement résolues. Par exemple, la suppression d'un composant du catalogue devrait idéalement propager une alerte aux netlists et layouts qui l'utilisent.

7.3 Bilan personnel

Ce projet nous a permis de découvrir concrètement les avantages de l'approche IDM pour des domaines métier spécialisés. La formalisation des concepts en méta-modèles oblige à une réflexion approfondie sur le domaine, ce qui clarifie les idées et révèle des cas limites non anticipés initialement.

Le travail en groupe a été facilité par la séparation en trois méta-modèles : chaque membre pouvait travailler sur une partie sans bloquer les autres. Cependant, les phases d'intégration ont demandé une coordination accrue pour s'assurer de la compatibilité des références croisées.

Nous avons particulièrement apprécié la puissance de Sirius pour créer rapidement des éditeurs graphiques sans développement lourd. La courbe d'apprentissage est certes significative, mais une fois les concepts maîtrisés, la productivité est au rendez-vous.